

Programming and Data Structures

Complete MCQ-Oriented Notes

IBPS SO (IT Officer) Examination - Complete Preparation Series

100% Syllabus Coverage - MCQ-Oriented - Zero Filler

Compiled by Asabhya Maanav

Table of Contents

Programming and Data Structures - CIL MT Systems Complete Notes	7
How to use this book	7
1. C Language Basics	8
1.1 Structure of a C program	8
1.2 Compilation steps	8
1.3 Character set, tokens, identifiers, keywords	8
1.4 Comments and I/O basics	9
2. Data Types, Variables, Constants	10
2.1 Primitive types	10
2.2 Modifiers	10
2.3 Size and range; sizeof	10
2.4 Type qualifiers	10
2.5 Storage classes	10
2.6 Scope and lifetime	11
2.7 Constants, literals, enum	11
2.8 #define vs const; typedef	11
3. Operators and Expressions	12
3.1 Arithmetic operators	12
3.2 Relational and logical	12
3.3 Bitwise operators	12
3.4 Assignment and compound	12
3.5 Increment / decrement	12
3.6 Ternary and comma	12
3.7 Precedence and associativity	12
3.8 Type conversion	13
4. Control Flow	14
4.1 if / else-if ladder	14
4.2 switch-case	14
4.3 Loops: for, while, do-while	14
4.4 break, continue, goto	14
4.5 Nested loops and counting	14
5. Functions	15

5.1 Prototype vs definition	15
5.2 Call and return	15
5.3 Call by value	15
5.4 Call by reference using pointers	15
5.5 Scope/lifetime of locals	15
5.6 Recursion	15
5.7 Function pointers	15
6. Pointers	16
6.1 Pointer concept	16
6.2 Null pointer	16
6.3 Pointer arithmetic	16
6.4 Pointers and arrays	16
6.5 Pointer to pointer	16
6.6 Pointers and const	16
6.7 void pointers	16
6.8 Dangling pointers and leaks	17
7. Arrays and Strings	18
7.1 1D arrays	18
7.2 2D / multidimensional arrays	18
7.3 Row-major address calculation	18
7.4 Arrays as function arguments	18
7.5 Array of pointers	18
7.6 Strings and the null terminator	18
7.7 Standard string functions	18
7.8 String manipulation	19
8. Structures and Unions	20
8.1 Structure basics	20
8.2 Nested structures and arrays	20
8.3 Pointers and self-referential structs	20
8.4 Structure size and padding	20
8.5 Unions	20
8.6 Bit-fields and typedef	20
9. Dynamic Memory Allocation	21
9.1 Memory layout	21
9.2 malloc, calloc, realloc, free	21

9.3 Allocating arrays dynamically	21
9.4 Common errors	21
10. File Handling and I/O	22
10.1 printf/scanf and format specifiers	22
10.2 Character/string I/O	22
10.3 File operations	22
10.4 EOF and command-line arguments	22
11. Preprocessor	23
11.1 #include and macros	23
11.2 Conditional compilation and guards	23
12. Recursion	24
12.1 Base case and recursive case	24
12.2 Recursion call stack	24
12.3 Direct vs indirect	24
12.4 Tail vs non-tail	24
12.5 Recursion vs iteration	24
12.6 Standard examples	24
12.7 Tracing recursion and counting calls	24
13. Abstract Data Type (ADT)	26
13.1 ADT vs data structure vs implementation	26
14. Arrays as a Data Structure	26
14.1 Static vs dynamic arrays	26
14.2 Operations and costs	26
14.3 Shifting cost	26
14.4 Address calculation	26
15. Stacks	28
15.1 Stack ADT and LIFO	28
15.2 Operations	28
15.3 Array vs linked-list implementation	28
15.4 Applications	28
16. Queues	29
16.1 Queue ADT and FIFO	29
16.2 Operations	29
16.3 Linear queue limitation	29

16.4 Circular queue	29
16.5 Linked-list queue	29
16.6 Dequeue	29
16.7 Priority queue	29
16.8 Applications	29
17. Linked Lists	30
17.1 Node structure	30
17.2 Singly linked list operations	30
17.3 Reversal and middle/nth node	30
17.4 Loop detection	30
17.5 Doubly linked list	30
17.6 Circular linked list	30
17.7 Array vs linked list	30
17.8 Applications	30
18. Tree Terminology	31
18.1 Core terms	31
18.2 Measures	31
19. Binary Trees	32
19.1 Definition and properties	32
19.2 Types	32
19.3 Counting relationships	32
19.4 Representation	32
19.5 Traversals	32
19.6 Height, size, leaf count, mirror	33
19.7 Expression trees	33
19.8 Threaded binary trees	33
20. Binary Search Trees (BST)	34
20.1 BST property	34
20.2 Search and insertion	34
20.3 Deletion	34
20.4 Min, max, floor, ceil	34
20.5 Inorder gives sorted order	34
20.6 Time complexity	34
21. Balanced Trees and Heaps	35
21.1 AVL trees	35

21.2 Red-Black trees (awareness)	35
21.3 B / B+ trees (concept)	35
21.4 Binary heap	35
22. Graph Terminology	37
22.1 Vertices and edges	37
22.2 Degree, path, cycle	37
22.3 Special graphs	37
23. Graph Representation	38
23.1 Adjacency matrix	38
23.2 Adjacency list	38
23.3 Trade-offs	38
24. Graph Traversal (DS view)	39
24.1 BFS	39
24.2 DFS	39
24.3 Connected components and cycle detection	39
25. Hash Tables	40
25.1 Hash table and hash function	40
25.2 Load factor	40
25.3 Collision resolution: chaining	40
25.4 Open addressing	40
25.5 Rehashing	40
26. Operation Cost Summary	41
26.1 Array / list / stack / queue	41
26.2 BST / AVL / heap	41
26.3 Hash table	41
27. CIL MT Exam Focus	42
27.1 C output prediction	42
27.2 Recursion tracing	42
27.3 Linked list correctness	42
27.4 Tree traversal / reconstruction	42
27.5 BST and heap sequences	42
27.6 Stack expression conversion	42
27.7 Complexity of standard operations	42

Programming and Data Structures - CIL MT Systems

Complete Notes

- This volume covers **Part A: Programming in C**, **Part B: Recursion**, **Part C: Linear Data Structures**, **Part D: Trees**, **Part E: Graphs**, **Part F: Hashing**, and **Part G: Complexity Summary**.
- The exam is an **objective MCQ Computer-Based Test (CBT)**; difficulty is conceptual-to-moderate, so master definitions, classifications, standard operations, output prediction, and short traces.
- Every red token is a likely answer key: memorise complexities such as **$O(1)$** , **$O(\log n)$** , **$O(n)$** , **$O(n \log n)$** , and **$O(n^2)$** .

How to use this book

- Read each "##" section in syllabus order; never skip the **CBT Trap** callouts because they list the exact confusions examiners exploit.
- Treat tables as memorisation targets; the **Operation Cost Summary** at the end consolidates every complexity.

1. C Language Basics

1.1 Structure of a C program

- A C program is a collection of **functions**; execution always begins at **main()** which returns **int** to the operating system.
- Standard skeleton parts: **preprocessor directives** (`#include`), **global declarations**, the **main() function**, and **user-defined functions**.
- A statement ends with a **semicolon (;)**; a block is a group of statements enclosed in **braces { }**.
- `#include <stdio.h>` brings in declarations for **printf** and **scanf**; angle brackets search system paths, quotes " " search the current directory first.
- The return value **0** from main conventionally signals **successful execution** (`EXIT_SUCCESS`); non-zero signals failure.

Memory Trick

- "PGMF": **Preprocessor, Globals, Main, Functions** is the top-to-bottom layout of a typical source file.

CBT Trap

- **main** returning void is non-standard; the portable signatures are **int main(void)** and **int main(int argc, char *argv[])**.
- `#include` lines have **no semicolon**; adding one is a common trick error.

1.2 Compilation steps

- The build pipeline has **4 stages**: **Preprocessing**, **Compilation**, **Assembly**, and **Linking**.
- **Preprocessing** expands `#include`, `#define`, and conditional directives, producing a **.i** (pure C) file.
- **Compilation** translates preprocessed C into **assembly (.s)** after syntax/semantic analysis.
- **Assembly** converts assembly into **machine code object file (.o / .obj)**.
- **Linking** combines object files plus libraries into the final **executable**, resolving external symbols.

Stage	Input	Output	Tool (gcc)
Preprocess	.c	.i	cpp
Compile	.i	.s	cc1
Assemble	.s	.o	as
Link	.o + libs	executable	ld

CBT Trap

- **Undefined reference** errors come from the **linker**, not the compiler; **syntax** errors come from the compiler.
- Macro expansion happens at **preprocessing**, before type checking, so macros are **not type-safe**.

1.3 Character set, tokens, identifiers, keywords

- A **token** is the smallest unit; C has **6 token classes**: **keywords**, **identifiers**, **constants**, **string literals**, **operators**, and **punctuators**.
- C89 has **32 keywords**; C99 added **5** (`inline`, `restrict`, `_Bool`, `_Complex`, `_Imaginary`) for **37**.
- Identifier rules: start with **letter or underscore**, followed by letters/digits/underscores; C is **case-sensitive**; cannot be a keyword.

- **Whitespace** and **comments** are not tokens; they only separate tokens.

CBT Trap

- `int` and `Int` are **different** identifiers because C is case-sensitive; only lowercase keywords are reserved.
- The number of C89 keywords (**32**) is a frequent direct MCQ.

1.4 Comments and I/O basics

- **Block comments** `/* . . . */` cannot be nested; **line comments** `// . . .` are C99 onward.
- Basic console output uses **`printf`**; input uses **`scanf`** with the **address-of (&)** operator for variables.

CBT Trap

- Forgetting `&` in `scanf("%d", &x)` causes **undefined behaviour** (writing to a garbage address), a classic output question.

2. Data Types, Variables, Constants

2.1 Primitive types

- Five basic types: **char**, **int**, **float**, **double**, and **void**.
- char** stores a single byte / character code; **int** stores integers; **float/double** store real numbers; **void** denotes "no type/value".
- Floating point follows **IEEE 754**: float is **32-bit single precision** (~6-7 significant digits), double is **64-bit double precision** (~15-16 digits).

2.2 Modifiers

- signed** / **unsigned** change sign interpretation; **short** / **long** change width.
- An unsigned **int** cannot be negative; its range is **0 to $2^n - 1$** .

2.3 Size and range; sizeof

- Typical 64-bit sizes: **char = 1**, **short = 2**, **int = 4**, **long = 8** (Windows 4), **float = 4**, **double = 8** bytes.
- sizeof** is a **compile-time operator** (not a function) yielding type **size_t**; **sizeof(char)** is guaranteed **1**.
- Signed n-bit range is **$-2^{(n-1)}$ to $2^{(n-1)} - 1$** ; for 32-bit int that is **-2,147,483,648 to 2,147,483,647**.

Type	Typical Bytes	Signed Range
char	1	-128 to 127
short	2	-32768 to 32767
int	4	-2.15e9 to 2.15e9
long long	8	-9.2e18 to 9.2e18

CBT Trap

- sizeof** of an array gives **total bytes**, but inside a function the array **decays to a pointer**, so **sizeof** gives **pointer size** (8 bytes on 64-bit), not array size.
- Standard guarantees only **minimum** sizes: **char** ≥ 8 bits, **int** ≥ 16 bits; exact sizes are implementation-defined.

2.4 Type qualifiers

- const** makes an object **read-only** (cannot be assigned after init); **volatile** tells the compiler the value may change unexpectedly, disabling caching/optimisation.
- volatile** is essential for **memory-mapped hardware registers** and variables modified by interrupts/other threads.

2.5 Storage classes

- Four storage classes: **auto**, **register**, **static**, **extern**.
- auto**: default for locals, stored on **stack**, lifetime = block.
- register**: hint to keep in **CPU register**; its **address cannot be taken** (no &).
- static**: lifetime = **whole program**; local static keeps value between calls; file-scope static gives **internal linkage**.
- extern**: declares a variable defined elsewhere (**external linkage**); no storage allocated by the declaration.

Class	Storage	Lifetime	Scope/Linkage	Default Init
auto	stack	block	block	garbage
register	register	block	block	garbage
static	data segment	program	block/file	0
extern	data segment	program	global	0

CBT Trap

- A **static local** variable is initialised **only once** and retains its value across calls - a favourite trace question.
- Global and static** variables are auto-initialised to **0**; **auto** (local) variables hold **garbage**.

2.6 Scope and lifetime

- Scope kinds: **block**, **function**, **file**, and **function-prototype**; lifetime kinds: **automatic** and **static**.
- Scope = where a name is **visible**; lifetime = how long the **storage exists**.

2.7 Constants, literals, enum

- Integer literal suffixes: **U** (unsigned), **L** (long), **LL** (long long); float **f** for float, **.0** defaults to double.
- enum** defines named integer constants starting at **0** by default and incrementing by **1**.

CBT Trap

- In enum {A, B=5, C}, values are **A=0, B=5, C=6** - explicit values reset the counter.

2.8 #define vs const; typedef

- #define is a **preprocessor text substitution** with **no type** and no scope; const is a **typed, scoped** variable honoured by the compiler/debugger.
- typedef** creates an **alias** for an existing type; it does **not** create a new type.

Feature	#define	const
Handled by	preprocessor	compiler
Type checking	no	yes
Scope	none (textual)	block/file
Memory	usually none	may have storage

CBT Trap

- #define SQ(x) x*x then SQ(2+3) expands to **2+3*2+3 = 11**, not 25 - always **parenthesise macro arguments**.

3. Operators and Expressions

3.1 Arithmetic operators

- Operators: **+** **-** ***** **/** **%**; **integer division truncates** toward zero; **modulo %** works only on **integers**.
- 7/2 is **3** (int), 7%2 is **1**, 7.0/2 is **3.5** (one operand float promotes the other).

3.2 Relational and logical

- Relational: **<** **<=** **>** **>=** **==** **!=**; logical: **&&** **||** **!**; result is **1 (true)** or **0 (false)**.
- Short-circuit**: **&&** stops at first **false**, **||** stops at first **true**; the right operand may never execute.

CBT Trap

- In `if (a && b++)`, if `a` is **0** then `b++` **never runs** - the postfix side effect is skipped.
- `=` is assignment, `==` is comparison; `if (x=5)` is always **true** (assigns 5) - a classic bug.

3.3 Bitwise operators

- Bitwise set: **&** (**AND**), **|** (**OR**), **^** (**XOR**), **~** (**NOT**), **<<** (**left shift**), **>>** (**right shift**).
- Set a bit: `x | (1<<k)`; clear a bit: `x & ~(1<<k)`; toggle a bit: `x ^ (1<<k)`; test a bit: `(x >> k) & 1`.
- Left shift by `k` multiplies by **2^k**; right shift divides by **2^k** (for unsigned/non-negative).
- XOR** of a number with itself is **0**; XOR with 0 leaves it unchanged - used to swap without a temp.

CBT Trap

- <<** has **lower precedence** than **+**; `1<<2+3` equals `1<<5` = 32, not 7.
- Right-shift of **negative signed** values is **implementation-defined** (arithmetic vs logical).

3.4 Assignment and compound

- Compound forms: **+=** **-=** ***=** **/=** **%=** **&=** **|=** **^=** **<<=** **>>=**; `a += b` means `a = a + b`.
- Assignment is an **expression** returning the assigned value and is **right-associative**, so `a=b=c` assigns `c` to `b` then to `a`.

3.5 Increment / decrement

- Prefix ++x** increments then yields the new value; **postfix x++** yields the old value then increments.
- In `y = x++`, `y` gets the **old** `x`; in `y = ++x`, `y` gets the **new** `x`.

CBT Trap

- Expressions like `i = i++ + ++i` are **undefined behaviour** (multiple unsequenced modifications); never predict a "right" answer except **"undefined"**.

3.6 Ternary and comma

- Ternary cond ? a : b is the only **three-operand** operator and is an **expression**.
- The **comma operator** evaluates left to right and yields the **rightmost** value; `(a, b)` has value `b`.

3.7 Precedence and associativity

- Highest to lowest (abridged): `() [] -> . > unary ! ~ ++ -- (cast) * & > * / % > + - > << >> > relational > == != > & ^ | > && || > ?: > assignment > ,`.
- Most binary operators are **left-associative**; **assignment**, **unary**, and **?:** are **right-associative**.

CBT Trap

- **Unary** operators outrank arithmetic; **== and !=** outrank **bitwise & ^ |**, so `a & b == c` parses as `a & (b==c)`.

3.8 Type conversion

- **Implicit promotion** (usual arithmetic conversions) widens smaller types toward the larger operand: **char/short -> int -> unsigned -> long -> float -> double**.
- **Explicit cast** `(type)expr` forces conversion and may **truncate** or lose precision.

CBT Trap

- `5/2*2.0` evaluates left to right: `5/2 = 2 (int)` then `*2.0 = 4.0`, **not 5.0**.

4. Control Flow

4.1 if / else-if ladder

- `if` tests a **non-zero** condition as true; an **else** binds to the **nearest unmatched if** (dangling-else rule).
- The **else-if ladder** checks conditions top-down and executes the **first** matching block.

CBT Trap

- A misplaced `;` as in `if(x>0);` makes an **empty** statement; the following block runs **unconditionally**.

4.2 switch-case

- `switch` compares an **integer/char constant** expression; cases need **break** to avoid **fall-through**.
- **default** runs when no case matches and may appear anywhere; case labels must be **compile-time integer constants**.

CBT Trap

- Missing **break** causes **fall-through**, executing subsequent cases; float/string cannot be a switch expression.

4.3 Loops: for, while, do-while

- **for** and **while** are **entry-controlled** (test before body); **do-while** is **exit-controlled** and runs the body **at least once**.
- A `for(;;)` or `while(1)` is an **infinite loop**.

Loop	Tested	Min Executions
for	before	0
while	before	0
do-while	after	1

4.4 break, continue, goto

- **break** exits the **innermost** loop/switch; **continue** skips to the **next iteration**; **goto** jumps to a **label** in the same function.
- `continue` in a `for` loop still executes the **update** expression.

CBT Trap

- **break** exits only **one** level; to leave nested loops you need a flag or **goto**.

4.5 Nested loops and counting

- Total iterations of two nested loops running n and m times is **$n \times m$** .
- Counting print statements / iterations is a standard MCQ; carefully track **off-by-one** boundaries.

5. Functions

5.1 Prototype vs definition

- A **prototype/declaration** specifies **return type, name, parameter types** and ends with `;;`; the **definition** provides the body.
- Prototypes enable **type checking** of calls before the definition appears.

5.2 Call and return

- A function returns **at most one** value via **return**; `void` returns none.
- The **return** statement also **terminates** the function immediately.

5.3 Call by value

- C passes arguments **by value** by **default**: the function gets a **copy**, so changes do **not** affect the caller's variable.

5.4 Call by reference using pointers

- To modify a caller's variable, pass its **address** and dereference inside: `void f(int *p){ *p = 5; }`.
- A classic **swap** works only with pointers: `swap(&a, &b)`.

CBT Trap

- Passing a variable normally cannot change it in the caller; you must pass a **pointer** (C has **no true pass-by-reference**, only pointer emulation).

5.5 Scope/lifetime of locals

- Locals/parameters live on the **stack**; they are created on **entry** and destroyed on **return**.

5.6 Recursion

- Covered fully in **Part B (Section 12)**.

5.7 Function pointers

- A function pointer stores a function's **address**: `int (*fp)(int, int);` then `fp = &add;` and call `fp(2, 3)`.
- Used for **callbacks**, **qsort** comparators, and **jump tables**.

CBT Trap

- `int (*fp)(int)` is a **pointer to function**; `int *fp(int)` is a **function returning int*** - parentheses change meaning.

6. Pointers

6.1 Pointer concept

- A **pointer** is a variable that stores a **memory address**; & is **address-of**, * is **dereference/indirection**.
- `int *p = &x;` makes p point to x; *p accesses x's value.

6.2 Null pointer

- A **NULL** pointer points to **no object** (value **0**); dereferencing it is **undefined behaviour** (usually a crash).
- Always initialise pointers; an uninitialised pointer is a **wild pointer**.

6.3 Pointer arithmetic

- Adding 1 to a pointer advances by **sizeof(pointed type)** bytes (**scaling**); `p+1` on `int*` moves **4** bytes.
- Valid operations: **pointer + integer**, **pointer - integer**, **pointer - pointer** (gives element count); **pointer + pointer is illegal**.

CBT Trap

- `p - q` returns the **number of elements** between them (type **ptrdiff_t**), not a byte count.

6.4 Pointers and arrays

- An array name **decays** into a pointer to its **first element** in most expressions; `a[i]` is equivalent to `*(a+i)`.
- Therefore `a[i] == *(a+i) == *(i+a) == i[a]` (commutative indexing).

CBT Trap

- An array name is **not** a modifiable lvalue: `a++` is **illegal**, but a pointer `p` can do `p++`.
- `&a` (whole array) has a different type (**pointer to array**) than `a` (pointer to element), though same address.

6.5 Pointer to pointer

- **Double indirection** `int **pp2` stores the address of a pointer; dereferencing twice (* then *) reaches the final value.
- Used for **dynamic 2D arrays** and to modify a pointer inside a function.

6.6 Pointers and const

- `const int *p` -> pointer to **const data** (data read-only, pointer movable).
- `int *const p` -> **const pointer** (pointer fixed, data writable).
- `const int *const p` -> both fixed.

Memory Trick

- Read right-to-left: "**const right of *** = const pointer; **const left of *** = const data".

6.7 void pointers

- A **void *** is a **generic pointer** that can hold any object address but **cannot be dereferenced** or incremented without a **cast**.

- malloc returns **void ***.

6.8 Dangling pointers and leaks

- A **dangling pointer** references **freed/out-of-scope** memory; a **memory leak** is allocated memory never **freed**.
- Best practice: set pointer to **NULL** after free.

CBT Trap

- Returning the address of a **local** variable creates a **dangling pointer** (the stack frame is gone after return).

7. Arrays and Strings

7.1 1D arrays

- Declaration `int a[5]` reserves **5** contiguous ints; indices run **0 to n-1**.
- C does **not** perform **bounds checking**; out-of-range access is **undefined behaviour**.

7.2 2D / multidimensional arrays

- `int a[r][c]` is stored as **r** rows of **c** elements in **row-major** order.
- A 2D array is an **array of arrays**; `a[i]` is the i-th row (itself an array).

7.3 Row-major address calculation

- For `a[i][j]` with base B, element size s, columns C: address = **$B + (i \cdot C + j) \cdot s$** (row-major).
- Column-major (FORTRAN) would be **$B + (j \cdot R + i) \cdot s$** ; C always uses **row-major**.

CBT Trap

- Address calculations are favourite numerals; confirm **row-major** and that index counting starts at **0** unless a lower bound is given.

7.4 Arrays as function arguments

- An array argument **decays** to a pointer; the function loses size info, so pass the **length** separately.

7.5 Array of pointers

- `char *arr[]` is an **array of pointers**, ideal for **ragged** string tables (argv-style).

7.6 Strings and the null terminator

- A C string is a **char array** terminated by **'\0'** (ASCII **0**); the terminator is what makes it a string.
- `char s[] = "Hi"` allocates **3** bytes: 'H','i','\0'.

CBT Trap

- `strlen("Hi")` is **2** but `sizeof("Hi")` is **3** (counts the **'\0'**).

7.7 Standard string functions

Function	Purpose	Notes
strlen	length excl. '\0'	O(n)
strcpy	copy src to dst	no bounds check
strcat	append	dst must have room
strcmp	compare	0 if equal
strchr	find char	returns pointer/NULL
strstr	find substring	returns pointer/NULL

- `strcmp` returns **<0, 0, >0** based on lexicographic order of the first differing character.

7.8 String manipulation

- **Reverse**: swap ends moving inward, **$O(n)$** ; **palindrome**: compare i and $n-1-i$; **count**: scan to `'\0'`.

CBT Trap

- `strcmp` returning **0** means **equal**; treating 0 as "false/not equal" is the classic mistake.

8. Structures and Unions

8.1 Structure basics

- A **struct** groups variables of **different** types under one name; members are accessed with **.** (**dot**) for objects and **->** (**arrow**) for pointers.
- `p->x` is shorthand for `(*p).x`.

8.2 Nested structures and arrays

- Structures can **nest**; an **array of structures** stores many records contiguously.

8.3 Pointers and self-referential structs

- A **self-referential structure** contains a pointer to its own type (`struct node *next`) - the basis of **linked lists and trees**.

8.4 Structure size and padding

- The compiler inserts **padding** for **alignment**, so `sizeof(struct)` may exceed the sum of member sizes.
- Members are aligned to their **natural boundary**; total size is padded to a multiple of the **largest member's alignment**.

CBT Trap

- Reordering members (largest first) can **reduce** padding; struct size is **>=** sum of members, never less.

8.5 Unions

- A **union** stores all members in the **same** memory; its size equals the **largest** member; only **one** member is valid at a time.

Feature	struct	union
Memory	sum + padding	largest member
Active members	all	one
Use	records	memory overlay / type-punning

8.6 Bit-fields and typedef

- **Bit-fields** pack members into a specified number of bits (`unsigned f:3;`) to save space.
- `typedef struct {...} Point;` lets you write `Point p;` without the `struct` keyword.

CBT Trap

- Writing one union member and reading another is **implementation-defined** (type punning); only the **last written** member is reliably valid.

9. Dynamic Memory Allocation

9.1 Memory layout

- A process image has **4** key regions: **text/code**, **data (initialised + BSS)**, **heap** (grows up), and **stack** (grows down).
- Dynamic** allocation uses the **heap**; locals use the **stack**.

9.2 malloc, calloc, realloc, free

- malloc(size)** allocates uninitialised bytes; **calloc(n,size)** allocates and **zero-initialises**; **realloc(p,size)** resizes a block; **free(p)** releases it.
- All allocators return **void *** (or **NULL** on failure) from `<stdlib.h>`.

Function	Initialises?	Args	Returns
malloc	no	(size)	void*
calloc	zeroes	(n, size)	void*
realloc	preserves	(ptr, size)	void*
free	n/a	(ptr)	void

9.3 Allocating arrays dynamically

- `int *a = malloc(n * sizeof(int));` creates a runtime-sized array; free with `free(a)`.

9.4 Common errors

- Memory leak**: forgetting free; **double free**: calling free twice; **use-after-free**: accessing freed memory - all **undefined behaviour**.

CBT Trap

- `calloc` initialises to **0**; `malloc` does **not** (contains garbage) - a frequent distinction MCQ.
- After `realloc`, the **old pointer may be invalid**; always use the returned pointer.

10. File Handling and I/O

10.1 printf/scanf and format specifiers

- Specifiers: **%d** int, **%u** unsigned, **%f** float/double-in-printf, **%lf** double-in-scanf, **%c** char, **%s** string, **%x** hex, **%o** octal, **%p** pointer, **%%** literal percent.
- printf returns the **number of characters printed**; scanf returns the **number of items successfully read**.

10.2 Character/string I/O

- **getchar/putchar** handle single characters; **fgets** reads a line safely (with size limit); **puts** writes a string and adds a **newline**.

CBT Trap

- **gets** is **unsafe/removed** (buffer overflow); use **fgets**. fgets keeps the trailing **"\n"**.

10.3 File operations

- **fopen(name, mode)** returns a **FILE *** (or **NULL** on failure); **fclose** flushes and closes.
- Modes: **"r"** read, **"w"** write (truncate/create), **"a"** append, plus **"b"** binary and **"+"** update.
- **fread/fwrite** do binary block I/O; **fprintf/fscanf** do formatted file I/O.

Mode	Meaning	If file missing
r	read	fails (NULL)
w	write (truncate)	creates
a	append	creates
r+	read/write	fails

10.4 EOF and command-line arguments

- **EOF** is a macro (commonly **-1**) signalling end of file; feof confirms it.
- main(int argc, char *argv[]): **argc** = argument count (≥ 1), **argv[0]** = **program name**, arguments follow.

CBT Trap

- **argc** is always ≥ 1 because **argv[0]** holds the program name; argv[argc] is **NULL**.

11. Preprocessor

11.1 #include and macros

- **#include** performs **file inclusion**; **object-like** macros `#define PI 3.14` substitute text; **function-like** macros `#define MAX(a,b) ((a)>(b)?(a):(b))` take arguments.
- Macros are expanded **textually** with **no type checking** and no function-call overhead.

11.2 Conditional compilation and guards

- **#ifdef** / **#ifndef** / **#endif** include code conditionally; an **include guard** prevents double inclusion:
- Pattern: `#ifndef H / #define H / ... / #endif`, equivalently `#pragma once`.

CBT Trap

- Always wrap macro bodies and arguments in **parentheses**; unparenthesised macros cause precedence bugs (see SQ example in 2.8).

12. Recursion

12.1 Base case and recursive case

- **Recursion** is a function calling **itself**; it needs a **base case** (stopping condition) and a **recursive case** that moves toward the base.
- Missing/incorrect base case causes **infinite recursion** -> **stack overflow**.

12.2 Recursion call stack

- Each call pushes an **activation record/stack frame** (parameters, locals, return address) onto the **call stack**; returns pop them.
- Maximum simultaneous frames = **recursion depth**, giving **O(depth)** stack space.

12.3 Direct vs indirect

- **Direct** recursion: a function calls **itself**; **indirect/mutual** recursion: f calls g and g calls f.

12.4 Tail vs non-tail

- **Tail recursion**: the recursive call is the **last** operation; it can be optimised into a **loop** (**tail-call optimisation**) using **O(1)** stack.
- **Non-tail**: work remains after the call (e.g., factorial multiplies after returning).

12.5 Recursion vs iteration

- Recursion is **cleaner** for self-similar problems but adds **call overhead** and **stack** usage; iteration is **faster** and uses **O(1)** space.

Aspect	Recursion	Iteration
Space	O(depth) stack	O(1)
Speed	call overhead	faster
Clarity	natural for trees	manual stack

12.6 Standard examples

- **Factorial**: $n! = n * (n-1)!$, base $0! = 1$; complexity **O(n)**.
- **Power**: fast exponentiation x^n in **O(log n)** by squaring.
- **GCD (Euclid)**: $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$, base $\text{gcd}(a, 0) = a$; **O(log min(a,b))**.
- **Fibonacci** (naive): **O(2^n)** time, exponential due to recomputation; memoised is **O(n)**.
- **Tower of Hanoi**: minimum moves = **2^n - 1**; recurrence $T(n) = 2T(n-1) + 1$.

Memory Trick

- Hanoi moves for n disks = **2^n - 1** (3 disks -> **7** moves).

12.7 Tracing recursion and counting calls

- Naive Fibonacci $\text{fib}(n)$ makes **2*fib(n+1) - 1** total calls; counting calls is a top MCQ.
- For $\text{fib}(5)$, the number of calls is **15** (since $\text{fib}(6) = 8$, $2 * 8 - 1 = 15$).

CBT Trap

- For naive Fibonacci, total calls to compute `fib(n)` equals $2 \cdot F(n+1) - 1$ where F is the Fibonacci number; do not confuse "calls" with "additions".

13. Abstract Data Type (ADT)

13.1 ADT vs data structure vs implementation

- An **ADT** is a **logical** description: a set of **values** plus **operations**, defined by **what** it does, hiding **how**.
- A **data structure** is a **concrete** way to organise data in memory; an **implementation** is the actual code.
- Example: **Stack ADT** (push/pop) can be implemented by an **array** or a **linked list**.

Level	Concern	Example
ADT	what (interface)	List, Stack, Queue
Data structure	how (organisation)	array, linked list
Implementation	code	C functions

CBT Trap

- **Stack/Queue/List** are **ADTs**; **array/linked list** are **data structures** - examiners swap these labels.

14. Arrays as a Data Structure

14.1 Static vs dynamic arrays

- **Static** arrays have **fixed compile-time** size on the **stack**; **dynamic** arrays are sized at **runtime** on the **heap** via **malloc** and can **realloc**.

14.2 Operations and costs

- **Access by index**: **O(1)** (random access via address arithmetic).
- **Search** (unsorted): **O(n)**; sorted with binary search: **O(log n)**.
- **Insert/Delete** at arbitrary position: **O(n)** due to **shifting**; at the **end** (amortised, dynamic): **O(1)**.

Operation	Array Cost
Access a[i]	O(1)
Search (unsorted)	O(n)
Insert at end	O(1) amortised
Insert at front	O(n)
Delete at front	O(n)

14.3 Shifting cost

- Inserting at index i requires shifting $n-i$ elements right; deleting shifts $n-i-1$ left.

14.4 Address calculation

- 1D a[i] with lower bound L: address = **B + (i - L)*s**.
- 2D row-major a[i][j]: **B + ((i-L1)*C + (j-L2))*s** where C = number of columns.

CBT Trap

- Arrays give $O(1)$ access but $O(n)$ insert/delete; linked lists are the opposite - know which wins for each operation.

15. Stacks

15.1 Stack ADT and LIFO

- A **stack** is a **LIFO** (Last-In-First-Out) structure; insertions/removals occur only at the **top**.

15.2 Operations

- push** (add to top), **pop** (remove top), **peek/top** (read top), **isEmpty**, **isFull**.
- All core operations are **O(1)**.

15.3 Array vs linked-list implementation

- Array** stack: a top index; **overflow** when $\text{top} = \text{capacity} - 1$, **underflow** when $\text{top} = -1$.
- Linked-list** stack: push/pop at the **head**; no fixed overflow but uses **extra pointer memory**.

Implementation	Push/Pop	Overflow	Extra memory
Array	O(1)	fixed capacity	none
Linked list	O(1)	only on heap exhaustion	pointer per node

15.4 Applications

15.4.1 Function call stack

- The **call stack** stores **activation records**; recursion depth maps to stack frames.

15.4.2 Balanced symbol checking

- Push opening brackets, pop and match on closing; balanced iff stack **empty at end** and never pops empty.

15.4.3 Infix to postfix/prefix

- Conversion uses an **operator stack** honouring **precedence** and **associativity**; postfix = **Reverse Polish Notation (RPN)**.

15.4.4 Postfix/prefix evaluation

- Evaluate postfix left-to-right: push operands; on an operator, pop **two**, apply, push result; final stack holds the **answer**.

15.4.5 Undo/backtracking

- Undo** stacks and **backtracking** (maze/DFS) rely on LIFO ordering.

Memory Trick

- "**LIFO** = a stack of plates"; you take the **top** plate first.

CBT Trap

- Postfix/prefix** expressions need **no parentheses**; **infix** does. Postfix evaluation pops the **second-popped** operand as the **left** operand for $-$ and $/$ (order matters).

16. Queues

16.1 Queue ADT and FIFO

- A **queue** is **FIFO** (First-In-First-Out): insert at the **rear**, remove from the **front**.

16.2 Operations

- enqueue** (add at rear), **dequeue** (remove from front), **front**, **rear**; all **O(1)**.

16.3 Linear queue limitation

- A simple array queue suffers **false overflow**: front advances leaving **unused** cells at the start that cannot be reused.

16.4 Circular queue

- A **circular queue** wraps indices with **modulo**: $\text{rear} = (\text{rear} + 1) \% \text{size}$.
- Full** condition: $(\text{rear} + 1) \% \text{size} == \text{front}$; **empty**: $\text{front} == \text{rear}$ (when one slot is left unused).
- A capacity-N circular queue using the "one empty slot" method holds **N-1** elements.

16.5 Linked-list queue

- Maintain **front** and **rear** pointers; enqueue at rear, dequeue at front, both **O(1)**.

16.6 Deque

- A **deque** (double-ended queue) allows insert/delete at **both** ends; specialisations are **input-restricted** and **output-restricted** dequeues.

16.7 Priority queue

- A **priority queue** dequeues the **highest (or lowest) priority** element, not FIFO; efficiently implemented with a **binary heap** giving **O(log n)** insert/extract.

16.8 Applications

- CPU/job scheduling**, **BFS** traversal, and **buffering** (I/O, print spooling) use queues.

Queue Type	Insert	Delete	Note
Linear	rear	front	false overflow
Circular	rear (mod)	front (mod)	reuses cells
Deque	both ends	both ends	flexible
Priority	by priority	highest priority	heap-based

CBT Trap

- In a circular queue, **empty** and **full** both make front and rear close; the **one-slot-unused** convention distinguishes them. Capacity stored = N but usable = **N-1**.

17. Linked Lists

17.1 Node structure

- A node holds **data** plus one or more **pointers**; built from a **self-referential struct** (`struct node{int d; struct node*next;}`).

17.2 Singly linked list operations

- Traversal: $O(n)$; search: $O(n)$; insert/delete at head: $O(1)$; insert/delete at tail: $O(n)$** (without tail pointer).
- Insertion needs only pointer rewiring - **no shifting**.

17.3 Reversal and middle/nth node

- Iterative **reversal** uses three pointers (prev, curr, next) in **$O(n)$** time, **$O(1)$** space.
- Middle** found by **slow/fast (tortoise-hare)** pointers in one pass.

17.4 Loop detection

- Floyd's cycle detection** uses slow (+1) and fast (+2) pointers; they **meet** inside a cycle; **$O(n)$** time, **$O(1)$** space.

17.5 Doubly linked list

- Each node has **prev** and **next**; allows **$O(1)$** deletion given a node and **bidirectional** traversal at the cost of an **extra pointer**.

17.6 Circular linked list

- The last node points back to the **first**; can be singly or doubly circular; useful for **round-robin** scheduling.

17.7 Array vs linked list

Aspect	Array	Linked List
Access by index	$O(1)$	$O(n)$
Insert/delete at front	$O(n)$	$O(1)$
Memory	contiguous	scattered + pointers
Resize	costly/fixed	dynamic
Cache locality	good	poor

17.8 Applications

- Polynomial representation** (term = coefficient + exponent + next), **sparse matrices**, and dynamic structures.

Memory Trick

- "Array = **seats in a row** (jump to any), List = **treasure hunt** (follow each clue)".

CBT Trap

- Linked list has **no random access**: reaching the k-th node is **$O(k)$** ; binary search is **not** efficient on a linked list.
- Singly list deletion needs the **previous** node; without it deletion is **$O(n)$** .

18. Tree Terminology

18.1 Core terms

- **Root**: top node (no parent); **node**: element; **edge**: link between two nodes; **parent/child**: adjacent up/down; **sibling**: same parent; **leaf**: node with **no children**.

18.2 Measures

- **Path**: sequence of edges; **depth** of a node = edges from **root** to it; **height** of a node = edges to the **deepest leaf**; **level** = depth + 1 (or depth, by convention); **degree** = number of **children**.
- **Subtree**: a node and its descendants; **ancestor/descendant**: up/down along a path.
- A tree with n nodes has exactly $n-1$ edges.

CBT Trap

- **Height of a single node** (or a leaf) is **0** (edge-count convention); **height of an empty tree** is **-1**. Depth of the root is **0**.

19. Binary Trees

19.1 Definition and properties

- A **binary tree** has each node with **at most 2** children (left/right).
- Max nodes at level i (root level 0) = 2^i ; max nodes in a tree of height h = $2^{(h+1)} - 1$.
- Minimum height of a binary tree with n nodes = $\text{floor}(\log_2 n)$; a tree with n nodes has height between $\log_2(n)$ and $n-1$.

19.2 Types

- **Full (strict)**: every node has **0 or 2** children.
- **Complete**: all levels full except possibly the last, filled **left to right**.
- **Perfect**: all internal nodes have 2 children and all leaves at the **same level**; has $2^{(h+1)}-1$ nodes.
- **Skewed/degenerate**: each node has **one** child (behaves like a **linked list**, height $n-1$).
- **Balanced**: height kept $O(\log n)$.

Type	Rule
Full	0 or 2 children
Complete	last level left-filled
Perfect	all leaves same level
Skewed	one child each (height $n-1$)

19.3 Counting relationships

- In a **full** binary tree with i internal nodes, leaves = $i + 1$.
- A binary tree with n nodes has exactly $n+1$ **NULL** pointers/links.
- Number of leaf nodes with degree-2 nodes count d_2 : **leaves = $d_2 + 1$** .

Memory Trick

- "**Leaves = internal-with-two-children + 1**" in a full binary tree.

19.4 Representation

- **Linked**: node with data + left + right pointers.
- **Array**: for index i (0-based), left child = $2i+1$, right child = $2i+2$, parent = $(i-1)/2$; efficient only for **complete** trees.

19.5 Traversals

19.5.1 Inorder

- **Inorder = Left, Root, Right**; for a **BST** it yields **sorted** order; complexity $O(n)$.

19.5.2 Preorder

- **Preorder = Root, Left, Right**; the **first** node is the **root**; good for **copying** a tree.

19.5.3 Postorder

- **Postorder = Left, Right, Root**; the **last** node is the **root**; used to **delete/free** a tree and evaluate **expression trees**.

19.5.4 Level-order (BFS)

- **Level-order** visits nodes level by level using a **queue**; complexity **O(n)**, space **O(width)**.

19.5.5 Reconstruction from traversals

- **Inorder + Preorder** or **Inorder + Postorder** uniquely rebuild a binary tree; **Preorder + Postorder** alone **cannot** (ambiguous) unless the tree is **full**.
- Inorder is **mandatory** for unambiguous reconstruction.

Traversal	Order	Root position
Inorder	L,Root,R	middle
Preorder	Root,L,R	first
Postorder	L,R,Root	last
Level-order	by level	first

19.6 Height, size, leaf count, mirror

- **Size** (node count), **height**, and **leaf count** are computed recursively in **O(n)**.
- **Mirror**: swap left and right subtrees recursively.

19.7 Expression trees

- Leaves are **operands**, internal nodes are **operators**; **inorder** gives **infix**, **postorder** gives **postfix**, **preorder** gives **prefix**.

19.8 Threaded binary trees

- (Awareness) **Threaded** trees replace NULL pointers with **in-order predecessor/successor** threads to enable **stackless** traversal.

CBT Trap

- **Inorder of a BST = sorted**; given preorder + postorder you **cannot** uniquely rebuild a general binary tree. Number of distinct binary trees with n nodes is the **Catalan number** $C(n) = (2n)! / ((n+1)! n!)$.

20. Binary Search Trees (BST)

20.1 BST property

- For every node: **left subtree** < **node** < **right subtree** (all keys), giving an ordered structure.

20.2 Search and insertion

- Search**: compare and go left/right; $O(h)$ where h is height ($O(\log n)$ balanced, $O(n)$ skewed).
- Insertion**: search to a **leaf** position and attach; $O(h)$.

20.3 Deletion

- Three cases: **leaf** (just remove); **one child** (replace with child); **two children** (replace with **inorder successor** (min of right subtree) or **predecessor** (max of left subtree)).

20.4 Min, max, floor, ceil

- Min** = leftmost node; **max** = rightmost node; **floor** = largest key $\leq x$; **ceil** = smallest key $\geq x$.

20.5 Inorder gives sorted order

- An **inorder** traversal of a BST outputs keys in **ascending** order.

20.6 Time complexity

Operation	Balanced	Skewed (worst)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

CBT Trap

- Inserting **sorted** data into a plain BST creates a **skewed** tree $\rightarrow O(n)$ operations; this is why **self-balancing (AVL)** trees exist.

21. Balanced Trees and Heaps

21.1 AVL trees

21.1.1 Balance factor

- **AVL** = self-balancing BST where **balance factor** = $\text{height}(\text{left}) - \text{height}(\text{right})$ is in $\{-1, 0, +1\}$ for every node.

21.1.2 Rotations

- Four rotation cases: **LL** (single right), **RR** (single left), **LR** (left-right double), **RL** (right-left double).
- A single rotation handles **LL/RR**; a double rotation handles **LR/RL**.

21.1.3 Insertion with rebalancing

- After insertion, walk up; the **first** node with balance factor **+/-2** is rebalanced with the matching rotation; AVL guarantees height **$O(\log n)$** .

Case	Imbalance	Fix
LL	left-left	single right
RR	right-right	single left
LR	left-right	left then right
RL	right-left	right then left

21.2 Red-Black trees (awareness)

- A **red-black** tree maintains balance via **5 properties**: nodes are **red/black**; **root is black**; **red node has black children** (no two reds adjacent); every root-to-NULL path has the **same number of black nodes** (black-height); all leaves (NIL) are **black**.
- Guarantees **$O(\log n)$** operations with **looser** balancing than AVL (fewer rotations on insert).

21.3 B / B+ trees (concept)

- **B-trees** are **multiway** balanced search trees used in **databases/file systems**; **B+ trees** keep all data in **leaves** linked for range queries (detailed in Databases).

21.4 Binary heap

21.4.1 Min/max-heap property

- A **binary heap** is a **complete** binary tree; **max-heap**: parent $\geq$ children; **min-heap**: parent $\leq$ children.

21.4.2 Array representation

- For index i (0-based): left = $2i+1$, right = $2i+2$, parent = $(i-1)/2$ (1-based: left $2i$, right $2i+1$, parent $i/2$).

21.4.3 Insert and delete-root

- **Insert**: add at end, **sift-up (bubble-up)**; **$O(\log n)$** .
- **Delete-root**: swap root with last, remove last, **sift-down (heapify)**; **$O(\log n)$** .

21.4.4 Build-heap and priority queue

- **Build-heap** from an array by sifting down from the last internal node is **$O(n)$** (not $O(n \log n)$).
- A heap implements a **priority queue**: **$O(1)$** peek-min/max, **$O(\log n)$** insert/extract.

21.4.5 Link to heap sort

- **Heap sort**: build a heap then repeatedly extract the root; **$O(n \log n)$** time, **$O(1)$** extra space, **not stable**.

Heap Operation	Complexity
Peek (min/max)	$O(1)$
Insert	$O(\log n)$
Extract root	$O(\log n)$
Build-heap	$O(n)$
Heap sort	$O(n \log n)$

Memory Trick

- " **$2i+1$** , **$2i+2$** " for 0-based children; " **$(i-1)/2$** " for parent.

CBT Trap

- A heap is **not sorted**; only the **root** is the extreme. **Build-heap** is **$O(n)$** , but heap sort overall is **$O(n \log n)$** . The smallest element in a **max-heap** is in some **leaf**, not at a fixed index.

22. Graph Terminology

22.1 Vertices and edges

- A **graph** $G = (V, E)$ is a set of **vertices** and **edges**; **directed** (edges have direction) vs **undirected**; **weighted** vs **unweighted**.

22.2 Degree, path, cycle

- **Degree** = edges incident on a vertex; directed graphs have **in-degree** and **out-degree**.
- **Path**: sequence of vertices via edges; **cycle**: path that returns to start.
- Sum of all degrees = $2 * |E|$ (Handshaking Lemma).

22.3 Special graphs

- **Connected**: path between every pair; **complete** (K_n): every pair adjacent with $n(n-1)/2$ edges (undirected); **bipartite**: vertices split into two sets with no intra-set edge; **tree**: connected acyclic with $n-1$ edges; **DAG**: directed acyclic graph.

CBT Trap

- A complete undirected graph K_n has $n(n-1)/2$ edges; a tree on n vertices has exactly $n-1$ edges; a graph is **bipartite** iff it has **no odd-length cycle**.

23. Graph Representation

23.1 Adjacency matrix

- An $n \times n$ matrix; entry $[i][j]=1$ (or weight) if edge exists; space $O(V^2)$; edge lookup $O(1)$.
- Symmetric for **undirected** graphs.

23.2 Adjacency list

- Array of lists, one per vertex listing neighbours; space $O(V + E)$; checking a specific edge is $O(\text{degree})$.

23.3 Trade-offs

Aspect	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V+E)$
Edge exists?	$O(1)$	$O(\text{degree})$
Iterate neighbours	$O(V)$	$O(\text{degree})$
Best for	dense graphs	sparse graphs

CBT Trap

- Use **adjacency list** for **sparse** graphs (saves space), **adjacency matrix** for **dense** graphs or $O(1)$ edge queries.

24. Graph Traversal (DS view)

24.1 BFS

- **BFS** explores level by level using a **queue** and a **visited** array; time **$O(V+E)$** (list), **$O(V^2)$** (matrix); finds **shortest path** in unweighted graphs.

24.2 DFS

- **DFS** explores as deep as possible using **recursion/stack** and a **visited** array; time **$O(V+E)$** .

24.3 Connected components and cycle detection

- A BFS/DFS from each unvisited vertex counts **connected components**; a **back edge** in DFS indicates a **cycle**.

CBT Trap

- **BFS** uses a **queue**; **DFS** uses a **stack/recursion** - mixing these up is a frequent trap. BFS gives shortest paths only in **unweighted** graphs.

25. Hash Tables

25.1 Hash table and hash function

- A **hash table** maps keys to **bucket indices** via a **hash function**, targeting **$O(1)$** average lookup/insert/delete.
- A good hash function is **deterministic**, **uniform**, and **fast**; common: **division method** $h(k)=k \bmod m$ and **multiplication method**.

25.2 Load factor

- **Load factor** $\alpha = n / m$ (entries / buckets); higher alpha increases **collisions**; for open addressing keep $\alpha < 1$, typically ≤ 0.7 before **rehashing**.

25.3 Collision resolution: chaining

- **Separate chaining** stores colliding keys in a **linked list** per bucket; search is **$O(1 + \alpha)$** average; allows $\alpha > 1$.

25.4 Open addressing

- All entries live in the **table itself**; probe sequences:
- **Linear probing**: $(h(k) + i) \bmod m$ - suffers **primary clustering**.
- **Quadratic probing**: $(h(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$ - reduces primary, has **secondary clustering**.
- **Double hashing**: $(h_1(k) + i \cdot h_2(k)) \bmod m$ - best distribution, **no clustering**.

Method	Formula	Clustering
Linear	$(h+i) \bmod m$	primary
Quadratic	$(h+c_1 i+c_2 i^2) \bmod m$	secondary
Double	$(h_1+i h_2) \bmod m$	none

25.5 Rehashing

- **Rehashing** builds a **larger** table (often **double** size) and reinserts all keys when the load factor exceeds a **threshold**; cost **$O(n)$** but **amortised $O(1)$** .

CBT Trap

- **Chaining** allows load factor > 1 ; **open addressing** cannot exceed **1**. **Linear probing** = primary clustering; **quadratic** = secondary clustering; **double hashing** avoids both.

26. Operation Cost Summary

26.1 Array / list / stack / queue

Structure	Access	Search	Insert	Delete
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Linked list	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
Stack	$O(n)$	-	$O(1)$	$O(1)$
Queue	$O(n)$	-	$O(1)$	$O(1)$

- * at head / given the node.

26.2 BST / AVL / heap

Structure	Search	Insert	Delete	Note
BST (avg)	$O(\log n)$	$O(\log n)$	$O(\log n)$	balanced
BST (worst)	$O(n)$	$O(n)$	$O(n)$	skewed
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	always balanced
Heap	$O(n)$ search	$O(\log n)$	$O(\log n)$	peek $O(1)$

26.3 Hash table

Case	Search	Insert	Delete
Average	$O(1)$	$O(1)$	$O(1)$
Worst	$O(n)$	$O(n)$	$O(n)$

Memory Trick

- "Balanced trees & heaps = $\log n$; hashing avg = $O(1)$; skewed/worst = $O(n)$ ".

27. CIL MT Exam Focus

27.1 C output prediction

- Watch **pointer arithmetic scaling**, **array decay**, **operator precedence**, **prefix/postfix**, and **integer division**.

27.2 Recursion tracing

- Count **calls** and **base-case** hits; remember naive Fibonacci does **$2 \cdot F(n+1) - 1$** calls.

27.3 Linked list correctness

- Verify **pointer rewiring order**; reversal uses **3 pointers**; deletion needs the **previous** node.

27.4 Tree traversal / reconstruction

- **Inorder** of BST is **sorted**; reconstruction needs **inorder** + one of pre/post.

27.5 BST and heap sequences

- Trace insertions/deletions; heap parent/child indices are **$2i+1$, $2i+2$, $(i-1)/2$** .

27.6 Stack expression conversion

- Convert/evaluate **infix/postfix/prefix**; postfix needs **no parentheses**.

27.7 Complexity of standard operations

- Memorise the **Operation Cost Summary** tables; distinguish **average vs worst** (BST, hashing).

CBT Trap

- The most common traps: **sizeof array vs pointer**, **strlen vs sizeof**, **$O(1)$ array access vs $O(n)$ list access**, **build-heap $O(n)$** , **BST worst case $O(n)$** , and **circular queue capacity $N-1$** .